PIPECIM: Energy-Efficient Pipelined Computing-in-Memory Computation Engine With Sparsity-Aware Technique

Yuanbo Wang, Liang Chang, *Member, IEEE*, Jingke Wang, Pan Zhao, *Student Member, IEEE*, Jiahao Zeng, Xin Zhao, *Student Member, IEEE*, Wuyang Hao, Liang Zhou, Haining Tan, Yinhe Han, *Senior Member, IEEE*, and Jun Zhou, *Senior Member, IEEE*

***Abstract*— Computing-in-memory (CIM) architecture has become a promising solution to improve the parallelism of the multiply-and-accumulation (MAC) operation for artificial intelligence (AI) processors. Recently, revived CIM engine partly relieves the memory wall issue by integrating computation in/with the memory. However, current CIM solutions still require large data movements with the increase of the practical neural network model and massive input data. Previous CIM works only considered computation without concern for the memory attribute, leading to a low memory computing ratio. This article presents a static-random access-memory (SRAM)-based digital CIM macro supporting pipeline mode and computation-memory-aware technique to improve the memory computing ratio. We develop a novel weight driver with fine-grained ping-pong operation, avoiding the computation stall caused by weight update. Based on our evaluation, the peak energy efficiency is 19.78 TOPS/W at the 22-nm technology node, 8-bit width, and 50% sparsity of the input feature map.**

***Index Terms*— Computing-in-memory (CIM), digital circuit, low power, multiply-and-accumulator, pipeline computing.**

I. Introduction

THE emergence of the deep neural network is a milestone for various artificial intelligence (AI) tasks, such as image classification, object detection, and automatic driving [1], [2], [3]. Recently, many large and complex neural networks have been developed to improve accuracy and intelligence, including Transformer and ChatGPT algorithms [4], [5]. However, with the increased number of parameters in these models, the computational complexity has also increased.

A potential solution for energy-efficient AI computation involves specialized processors and accelerators with optimized processing elements (PEs) and architecture to accelerate convolutional operations [6], [7]. However, the data movement issue remains a bottleneck for these devices. The computing-in-memory (CIM) architecture, which performs multiply-and-accumulation (MAC) operations inside/with the memory, is recognized as a promising solution to solve the "memory wall" issue induced by the data movements. While static random access memory (SRAM) is an established memory technology that can be utilized to develop high-performance AI processors [8], the density and scalability limitations of SRAM-based CIM macros restrict their ability to process large neural network models. Hence, addressing the "memory wall" issue and carefully considering parameter updating between the CIM macro and off-chip memory is crucial.

When it comes to handling AI tasks, we observe three weaknesses of the SRAM-CIM-based AI processor: weight update latency, which potentially stalls computation; increased data buffer requirements, compromising computation efficiency; and underutilization of neural network sparsity. Several studies have proposed solutions to address the above weaknesses. For instance, the ping-pong structure in analog CIM enables simultaneous computation and weight updating on the CIM macro [8]. In addition, the ping-pong buffer offers individual weight SRAM buffers for updating weights [9], [10]. Various works have also reported measured results at different levels of neural network sparsity [11], [12].

This article proposes a digital pipeline CIM compute engine for AI processors. Through a series of pipeline operations, we successfully address compute stalls caused by various factors, particularly weight updates as mentioned earlier. In addition, we propose a comprehensive sparsity mapping scheme to enhance the performance of the neural network on this engine and improve the energy efficiency of the entire system. The measured energy efficiency of CIM macro is 6.92∼80.9 TOPS/W for INT8 at different sparsity levels. The energy efficiency of the overall system is 8.68∼25.67 TOPS/W for INT8 and 2.35∼7.30 TFPS/W for BF16 at VGG-16. The main contributions of this article are given as follows.

1) We propose a novel ping-pong SRAM structure that supports simultaneous computation and weight updates.

Received 30 March 2024; revised 9 July 2024 and 11 August 2024; accepted 23 August 2024. Date of publication 1 October 2024; date of current version 23 January 2025. This work was supported in part by the National Natural Science Foundation of China under Grant 62104025 and in part by the Houmo Research Funding under Grant 202205FKY00172. *(Corresponding authors: Liang Chang; Haining Tan.)*

Yuanbo Wang, Liang Chang, Jingke Wang, Pan Zhao, Jiahao Zeng, Xin Zhao, Wuyang Hao, Liang Zhou, and Jun Zhou are with the School of Information and Communication Engineering, University of Electronic Science and Technology of China, Chengdu 611731, China (e-mail: liangchang@uestc.edu.cn).

Haining Tan and Yinhe Han are with the Institute of Computing Technology, Chinese Academy of Sciences, Beijing 100190, China (e-mail: tanhaining@ict.ac.cn).

Digital Object Identifier 10.1109/TVLSI.2024.3462507

1063-8210 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:41:51 UTC from IEEE Xplore. Restrictions apply.

Compared to the static weight storage method, we alleviate the computation stalls caused by the weight updates, eliminating the need for individual weight SRAM buffers. This improvement enhances energy efficiency in weight updates while achieving higher area efficiency.

2) A pipeline CIM architecture has been proposed, which eliminates the other factors that may cause computation stall. For example, loading input features and output convolution results for activation. The floating-point number can also be supported at the architecture level.
3) A comprehensive zero-skipping and clock-gating scheme directly skips operations based on the sparsity of the neural network data distribution. This approach enhances the performance of MAC operations and improves the overall system energy efficiency. An analysis has been conducted to evaluate the impact of neural network sparsity and CIM macro parallelism on both throughput and energy efficiency.

The rest of this article is organized as follows. Section II provides CIM computation, weight update issues, and the sparsity of neural networks. Section III illustrates the PIPECIM architecture and the detailed sparsity mapping scheme. Section IV shows the proposed CIM macro. Section V demonstrates the experimental results and the corresponding analyses. Section VI concludes this article.

II. Preliminary and Motivation

In this section, the drawbacks of current CIM architecture are analyzed. Furthermore, we discuss the weight update and sparsity issues of the current CIM architecture.

A. Necessary Macro Weight Update

Typically, the parameter size of neural networks exceeds the capacity of on-chip SRAM. Taking ResNet-50 as an example, it requires 23.4549 million convolution parameters and 2.048 million fully connected layer parameters, which far exceeds the capacity of the on-chip CIM macro. Consequently, the majority of neural network weights are stored in the off-chip memory, while the computation data are only loaded and updated in on-chip buffers and the CIM macro.

One issue with the above scheme is the computation stall caused by weight updates. In a basic 6-T SRAM cell, both read and write operations require one cycle. During this cycle, there are various steps, such as precharging BL and BLB, decoding addresses, and enabling the write buffer (for write operations) or sense amplifier (for read operations). Weight updates are performed by write operation. If multiple SRAM cells ($M \geq 1$) sharing the same BL and BLB need to be written, M cycles are needed. When writing to an SRAM cell, the value stored in it becomes uncertain. If the units to be written also need to be read out as the inputs for compute circuit, the computation must be stalled at that time until the value in the cell is certain again.

Considering weight updates, the previous SRAM-CIM approach exhibited significant drawbacks. The previous weight storage method, such as static storage, results in computation stalls of 256 cycles, 128 cycles, and 384 cycles [11], [12], [13] due to the weight updates. Using fixed weight is the primary design strategy in CIM-based AI accelerators. The input activation (INA) of the compute circuit changes each cycle, while the weight only changes after being maximally reused. Ideally, weight changes should occur after computing a complete input feature map. However, this approach results in the need for increased buffer size to store intermediate partial accumulated results.

TABLE I
Compute/Weight Update Ratio of [11]

Layer	Output Size	Update Time (Cycle)	Compute Time (Cycle)	Ratio	Buffer Size (Scale to Conv5)
Conv1	112x112	256	12544	49	256
Conv2	56x56	256	3136	12.25	64
Conv3	28x28	256	784	3.0625	16
Conv4	14x14	256	196	0.7656	4
Conv5	7x7	256	49	0.1914	1

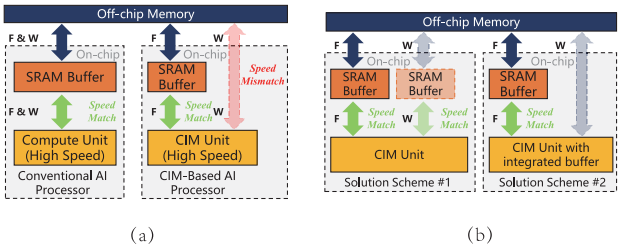


Fig. 1. (a) Illustration of speed mismatch. (b) Two solution schemes of speed mismatch.

To illustrate this issue, we consider the implementation of ResNet-50 [2] on the architecture proposed in [11] for image classification using the ImageNet dataset. Table I provides the weight update time, maximal compute time, and partial sum buffer size of different layers. It can be observed that when the output size (feature map) is 112 × 112 and 56 × 56, the compute time to weight update ratio is more than 10. This indicates that, in these cases, weight updates have a negligible impact, but the large buffer size required for storing intermediate results remains a significant concern. When the output size is less or equal to 28 × 28, the influence of weight updates becomes significant. This effect is particularly pronounced when the output size is 7 × 7, where weight updates account for almost 80% of the execution time of the entire circuit, severely limiting the throughput of the CIM-based AI accelerator.

The other issue is the speed mismatch between the on-chip compute unit and off-chip memory, as shown in Fig. 1(a), and F and W represent the feature and weight, respectively. In conventional processors, F and W are preloaded into an SRAM buffer to hide this problem. However, in CIM-based AI accelerators, the weight is delivered directly from off-chip memory to the CIM unit, leading to speed mismatch. To address this issue, two solution schemes are proposed. The first scheme is to add a weight buffer between the compute unit and the off-chip memory. This approach eliminates the speed mismatch but undermines the architectural advantage of CIM. The second scheme is to give enough buffer capability

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:41:51 UTC from IEEE Xplore. Restrictions apply.

to the CIM unit while keeping the energy consumption in an acceptable range. This eliminates the need for individual weight buffers, making it a more reasonable solution.

B. Sparsity of Neural Networks

Based on our research, neural networks share the commonalities that the weight and feature often contain zero values or values close to zero. Our analysis is as follows.

1) The wide use of the ReLU activation function. ReLU forces all negative values of the output feature to zero. As for activation function which is not ReLU, the abs of negative output is small.
2) The natural result of training. Usually, the number of parameters is large, but only some are crucial. Some parameters are superfluous or have little effect on the final results, which will gradually decrease even to zero.
3) The use of quantization. Quantizing parameters to a lower precision (e.g., 8-bit integer) is a typical operation since float operations are energy-hungry. Many small values mentioned in 1) and 2) become zero in the quantization process.

Without advanced sparsity techniques, we have quantized two typical neural networks, VGG-16 and ResNet-50. The accuracy loss is less than 0.5% for VGG-16 and 4% for ResNet-50. The sparsity statistic of VGG-16 is illustrated in Fig. 2. We can conclude that the following conditions hold.

1) Abundant sparsity exists. The feature sparsity of convolution layers is 44.32%∼89.50% for ImageNet and 50.59%∼98.6% for CIFAR-10. The weight sparsity of convolution layers is 2.95%∼21.62% for ImageNet and 1.91%∼99.66% for CIFAR-10.
2) The deeper convolution layer has higher sparsity. The feature sparsity for ImageNet increases from 44.32% of Conv01 to 89.49% of Conv13.
3) Fully connected layer also has considerable sparsity. The feature and weight sparsity is 74.74% for FC1 and 60.36% for ImageNet. The line chart of the statistic results of ResNet-50 is not put up, whose structure is much more complex than that of VGG-16. The feature sparsity of ResNet-50 is 10.02%∼93.03% for ImageNet and 14.94%∼96.85% for CIFAR-10. In the ResNet-50 network structure, the feature sparsity of ImageNet ranges from 10.02% to 93.03%, while the feature sparsity of CIFAR-10 ranges from 14.94% to 96.85%. The weight sparsity is 5.91%∼21.35% for ImageNet and 5.92%∼93.02% for CIFAR-10.

The sparsity of neural networks can be used to significantly improve throughput and energy efficiency, but how to use it is a real challenge. Sticker [7] is an AI processor, which utilizes the dual sparsity of feature and weight to improve overall energy efficiency significantly. Furthermore, different modes can be adopted based on the sparsity of both feature and weight. For current CIM works, the sparsity of neural networks is considered. For example, the peak energy efficiency of [11], [14], [15] is measured under 18% input toggle rate, 50% input bit sparsity, and 90% input sparsity.

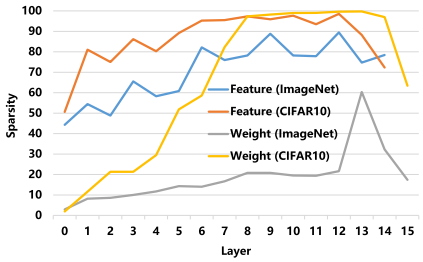


Fig. 2. Layer-wise sparsity of VGG-16.

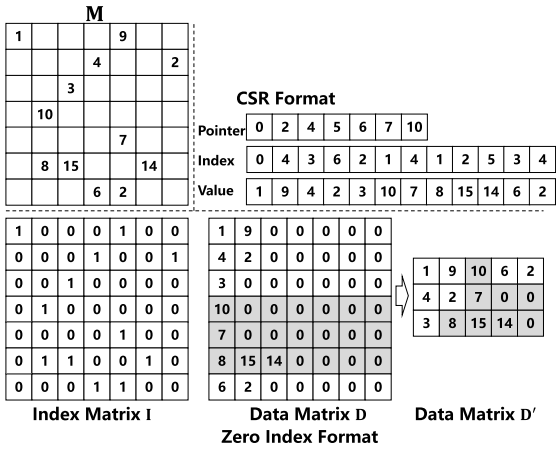


Fig. 3. Two encoding schemes of sparsity matrix: CSR and zero index.

Sparse matrix has different storage formats, such as compressed sparse row (CSR) [16], [17], coordinate (COO), and zero index [7]. COO only stores a sparse matrix's coordinates and values of nonzero elements. CSR and zero index are illustrated in Fig. 3. When matrix M is very sparse, COO or CSR is the best choice. When the sparsity of M is not abundant too much, the zero index format is a better choice since the memory space of each element of index matrix I only needs 1 bit.

III. PIPECIM Architecture and Design Components

A. Overall Architecture of PIPECIM

The proposed PIPECIM architecture, as shown in the right of Fig. 4, consists of a top controller, an INA preprocessing unit, an INA buffer, a 4×8 pipeline CIM macro array, 64 pipeline accumulators, and 64 postprocessing units. The top controller generates control signals based on instructions from the host and provides feedback on execution results. The INA preprocessing unit prepares the INAs of the neural network for efficient calculation in the CIM macro array. The macro array is composed of $4 \times 8 = 32$ CIM macros, each containing two CIM units. Each CIM unit can store nine 8-bit weights and perform accumulation operations with the INA, which is naturally well suited for 3×3 convolutions. In order to minimize the overhead of the INA preprocessing unit, all CIM units share the same INA. After the convolution operation, the results from the CIM unit are accumulated in the pipeline accumulator to obtain final results with increased

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:41:51 UTC from IEEE Xplore. Restrictions apply.

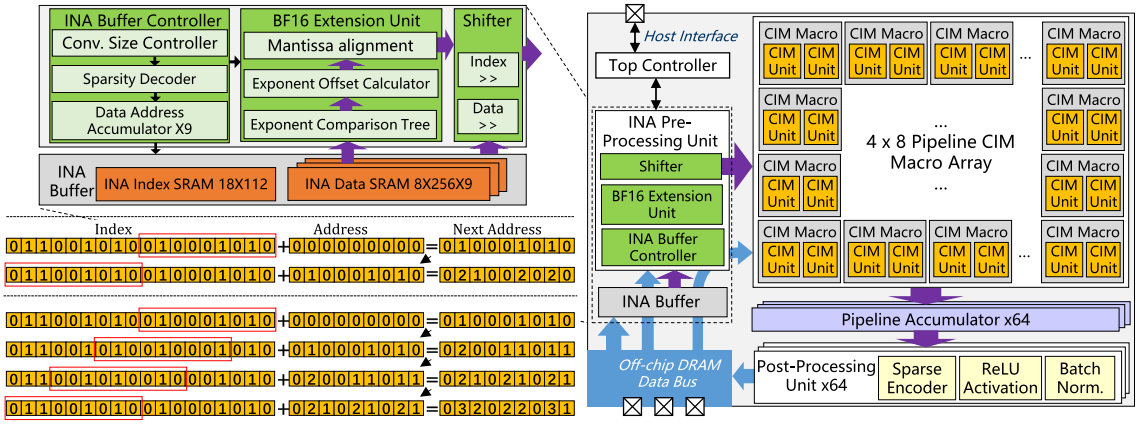


Fig. 4. Top left: detailed INA preprocessing unit. The middle left and bottom left are examples of how indices control the addresses of data SRAMs. Middle: Conv. 1 × 1. Bottom: Conv. 3 × 3. Right: proposed PIPECIM architecture.

input channels. Finally, the results are adjusted to INT8 or BF16 and sent to off-chip memory.

The INA preprocessing unit is illustrated in the top left of Fig. 4. It consists of three components: the INA buffer controller, the BF16 extension unit, and the shifter. The INA buffer controller determines the read and write scheme of the INA buffer based on the structure parameter of neural networks, such as the kernel size, the feature map size, and the sparsity of neural networks. The BF16 extension unit selects the computation precision (INT8 or BF16). The input feature of neural networks is temporally saved in the INA buffer, which is composed of an 18 × 112 INA index SRAM and nine 8 × 256 data SRAMs. All SRAMs are dual port SRAMs to enable simultaneous writing and reading. The 1 × 1 and 3 × 3 convolution kernels are widely used in neural networks. Therefore, we use these two types of convolution kernels as examples for mapping. Two different store schemes indicated in Fig. 5 are adopted to improve the utilization of the CIM macro array. Index tensor I denotes all indices of multiple input feature maps, while data tensor D' only denotes the nonzero activations.

B. INA Preprocessing Unit

For 1 × 1 convolution, the indices of every 18 input channels with the same position are stored in the INA Index SRAM. The nine nonzero activations of nine input channels are sequentially saved at different data SRAMs in ascending order of addresses. The addresses of nine data SRAMs are controlled by the indices output by index SRAMs and reset to zero when this compute engine starts to work. The indices will be valid in two cycles. In the first cycle, the high 9 bits of the indices are used to determine the address-increasing signal for the nine data SRAMs, while in the second cycle, the low 9 bits are used. The middle left of Fig. 4 presents an example. For the index $\{18'b011001010_010001010\}$, the Addr of data SRAMs will be $\{8'd0, 8'd1, 8'd0, 8'd0, 8'd0, 8'd1, 8'd0, 8'd1, 8'd0\}$ and $\{8'd0, 8'd2, 8'd1, 8'd0, 8'd0, 8'd2, 8'd0, 8'd2, 8'd0\}$ in the next two cycles.

For 3 × 3 convolution, the intrachannel INAs are first stored. Specifically, the adjacent 18 indices at three rows of index tensor I are combined into an index block; each block is saved in the same address of index SRAM. Three neighboring nonzero INAs at each column are repeatedly written to nine data SRAMs. Nine data SRAMs are divided into three groups, and the intragroup write addresses are the same, but the adjacent intergroup write addresses are incremental. Nine nonzero INAs can be read simultaneously in this way. Different from Conv. 1 × 1, the output indices will be valid in the next four cycles. An example is presented in the bottom left of Fig. 4, assuming that the output indices of index SRAM and initial addresses are the same as the above. Addr of nine data SRAMs will be $\{8'd0, 8'd1, 8'd0, 8'd0, 8'd0, 8'd1, 8'd08'd1, 8'd0\}$, $\{8'd0, 8'd2, 8'd0, 8'd0, 8'd1, 8'd1, 8'd0, 8'd1, 8'd1\}$, $\{8'd0, 8'd2, 8'd1, 8'd0, 8'd2, 8'd1, 8'd0, 8'd2, 8'd1\}$, and $\{8'd0, 8'd3, 8'd2, 8'd0, 8'd2, 8'd2, 8'd0, 8'd3, 8'd1\}$ in the next four cycles. The corresponding nonzero INAs are emitted synchronously. The final output of the INA preprocessing unit includes indices and nonzero INAs. Indices are converted to the clock enable signal of the CIM unit. Only the nonzero INAs participate in the real calculation, which will be further described in Section IV. However, our CIM macro supports not only these two types of convolution kernels but also other types of convolution kernels. For convolution kernels with a larger size, such as 7 × 7, it is decomposed to multiple 3 × 3 kernels and then calculated. Activations for stride = 2 are sent to the CIM macro array by stride = 1, but the unnecessary compute of CIM macro array is clock-gated. For input feature maps with high sparsity, the skip-zero operation is used to accelerate the process. For Conv. 1 × 1, the operation is skipped if the lower 9 bits of indices are all zeros, as the Conv. result is zero regardless of the weight. Thus, the high 9 bits are used to control Addr directly. For Conv. 3 × 3, the skipping operation will be executed as long as nine neighboring zero indices occur.

For BF16 precision, the storage format of INA is illustrated in Fig. 6. The INA preprocessing unit separately stores the exponent term and the mantissa term (along with the signed term) in data SRAM. Different from the 8-bit integer, the valid time of indices of BF16 will be doubled to load complete BF16 data from data SRAM. In the first circle, exponents are read out to the exponent comparison tree. In the second cycle,

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:41:51 UTC from IEEE Xplore. Restrictions apply.

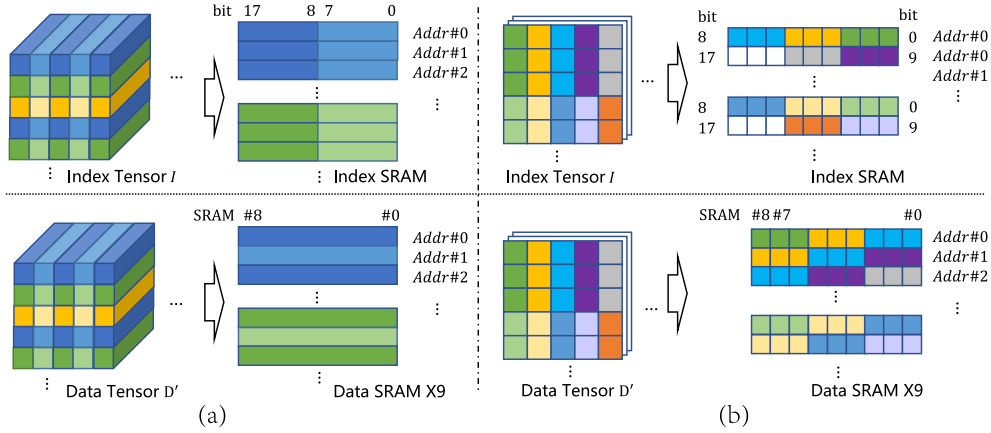


Fig. 5. Scheme of mapping input feature to INA buffer. (a) Conv. 1 × 1. (b) Conv. 3 × 3.

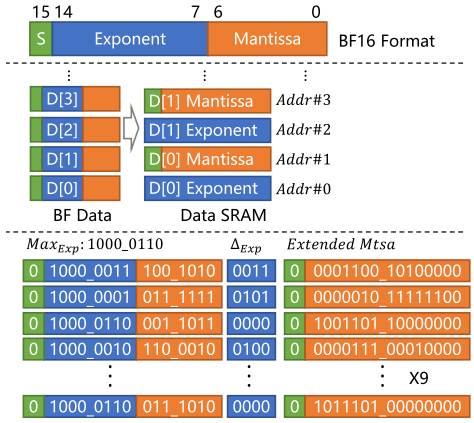


Fig. 6. Saving and operating scheme of BF16 data.

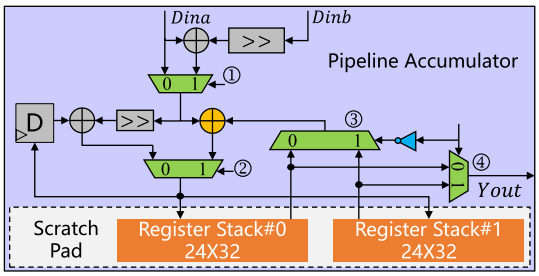


Fig. 7. Pipeline accumulator.

mantissas will be read to the mantissa alignment unit. The process scheme of BF16 is shifting the mantissa of INA and weight to 16 bits and then sending them to the CIM macro array in the next two cycles. The exponent comparison tree is used to find the maximal exponent and the exponent's offset of nine INAs by the exponent offset calculator synchronously. Then, the mantissa of nine INAs is aligned to 16 bits according to the exponent offsets in the mantissa alignment unit. Hidden bit 1 of BF16 is exposed in this process. The index and data shifter ensure that the index and data output are in the same cycle.

The bottom right of Fig. 6 has shown an example of BF16 alignment. Original 9 BF16 data are $\{16'b0_1000_0011_100_1010\}$, $\{16'b0_1000_0001_011_1111\}$, ..., $\{16'b0_1000_0110_011_1010\}$. The exponent comparison tree calculates the Max_{Exp}:$\{8'b1000_0110\}$. The exponent offsets are denoted by Δ_{Exp}. The extended 16-bit mantissa is converted to 2's complement and fed to the CIM macro array in two cycles, with the high 8 bits processed first. The convolution of the extended 16-bit mantissa is calculated by two CIM units, which will be further described in Section IV. Note that weights share the same mantissa extension scheme with INA but should be extended to 16 bits before being written to CIM units. High 8 bits and low 8 bits are placed in two neighboring CIM units.

C. Pipeline Accumulator

The pipeline accumulator is illustrated in Fig. 7. The scratch pad, composed of two 24 × 32 register stacks, is used to store partial accumulated results of the convolution operation, which will be reset to zero before the pipeline accumulator starts to work. When one register stack is used to accumulate, the convolution results from the other register stack are sent to the postprocessing unit to execute batch normalization (BN) and ReLU activation. This operation is controlled by Muxes 3 and 4. Therefore, the computation stalls caused by outputting final convolution results are avoided. Dina and Dinb are two 3 × 3 convolution results of CIM macro array. The dark components in the figure are used for BF16 calculation. For INT8 data precision, Muxes 1 and 2 turns to 0 and 1, in order to send the result of adding Dina and the initial partial sum to the scratch pad. For BF16 data precision, Dina is the calculation result of high 8 bits of weight and INA, while Dinb refers to low 8-bit weight. Therefore, only half of the pipeline accumulators works. Dina and shifted Dinb are shifted and added to 24 bits before Mux 1. If the corresponding INA of Dina and Dinb is high 8 bits, the accumulating results are written to D flip-flop temporally. The accumulated results will be written to scratch pad by turning Mux 2 to 0 when the low 8-bit calculation results are shifted and added in the next cycle.

D. Postprocessing Unit

The postprocessing unit is illustrated in Fig. 8. BN, ReLU activation, and the output feature map are executed serially.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:41:51 UTC from IEEE Xplore. Restrictions apply.

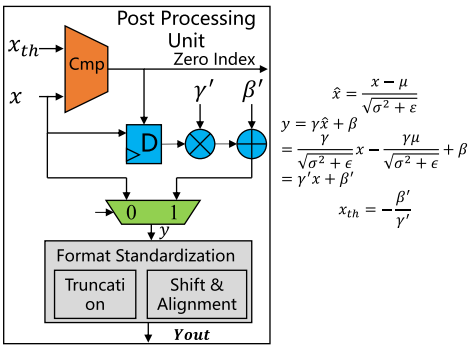


Fig. 8. Postprocessing unit.

γ' and β' are the combination of γ, β, μ, and σ of [18]. The look-ahead activation is executed to avoid unnecessary calculations. When $x \geq x_{\text{th}}$ and $y \geq 0$, BN is executed. Otherwise, it will be clock-gated. For the network without BN, such as VGG, x is sent to format standardization directly. The output features are truncated to 8-bit integers or converted to standard BF16 format in the format standardization module.

IV. Pipeline CIM Macro Design

A. Overall Structure of CIM Macro

The overall structure of the proposed digital CIM macro is shown in Fig. 9, which consists of two CIM units; each CIM unit includes an SRAM read/write controller, nine ping-pong SRAM blocks, nine pipeline multipliers, and a small-scale adder tree. The weights of the neural network are stored in the ping-pong SRAM block. The pipeline multiplier can multiply 8-bit weight and 8-bit INA. CKEN is the clock enable signal. The pipeline multiplier is clock-gated, and its output is tied to zero synchronously if INA or weight is zero by checking their index in advance. The adder tree is intertwined by 14T and 28T full adders reported in [11]. With the adder tree and nine pipeline multipliers, performing operations such as 3×3 convolutions is very convenient in macro.

B. Parallelism and Sparsity Consideration

In the design of CIM macro, a basic awareness is that more extensive scale and higher parallelism are beneficial. More circuits can enjoy the advantages of custom circuit design. The hybrid usage of 14T and 28T in [11] is a good instance. It is valid for dense matrices since no more operations can be used. They are rethinking whether the multiplication of zero weight or zero INA needs to be clock-gated. If all INAs of the nine pipeline multipliers in Fig. 9 are zero predicted in advance, clock gating is unnecessary. This convolution can be skipped directly. The skipping scheme improves the throughput of CIM macro and energy efficiency. For higher parallelism CIM macro with an adder tree, such as [11], the skipping operation can be valid only when 256 INAs are zero simultaneously. The probability of the latter is much less than the former. Less parallelism may be more efficient since the sparse matrix is standard in neural networks (see Fig. 2). For simplicity, nine parallelisms are adopted in this work. More proper parallelism can be found in practical applications.

C. Ping-Pong SRAM Block

The ping-pong SRAM block is illustrated in Fig. 10, divided into eight columns. Each column is composed of 32 SRAM cells. Therefore, 32 8-bit weights can be stored in one ping-pong SRAM block. The SRAM R/W controller controls all BL and BLB and is equipped with 72 sets of precharger, a write buffer, and a sense amplifier to perform the normal read and write operations of nine ping-pong SRAM blocks. Unlike previous CIM macro designs [9], [15], where computing is implemented in BL and BLB, the idle BL and BLB in this work allow both compute and SRAM access to work simultaneously. The same WL controls the cell at the same row of the different columns. The value stored in SRAM cell #0 and #1 is the output to pipeline multiplier through a 2-to-1 multiplexer, which can achieve a fine-grained ping-pong operation. When one SRAM cell is used to compute, the value stored in the other can be updated, which avoids the compute stall of CIM macro caused by weight update.

SRAM cells #2–#31 do not have a direct path to the pipeline multiplier. Their purpose is to increase the memory capacity of the CIM macro and provide enough buffer capacity to overcome the speed mismatch issue referred to in Section II. The individual weight SRAM buffer is discarded in this way. The initialization phase of the CIM macro can be triggered whenever SRAM cell #0 or #1 is written. The other SRAM cells can be written synchronously with compute.

The weight update of the proposed ping-pong SRAM block is energy-saving. For conventional AI processors [6] or in the prior CIM work [8], the weight update path is from individual SRAM buffer to compute unit or CIM macro. For simplicity, the individual SRAM buffer is called external SRAM (compared with compute unit or CIM macro) in the rest of this article. The update needs two steps: one is reading weight from the external SRAM and the other is writing to the compute unit in [6] or CIM macro in [8]. Reading the external SRAM and writing the CIM macro requires a two-stage process. First, precharge the BL and BLB, and then, activate the read amplifier of the CIM macro and the actual write of the external SRAM write buffer. The weight update path for our proposed ping-pong SRAM block is from SRAM cells #31∼#2 to SRAM cells #1 and #0. Reading SRAM cells #31∼#2 needs two phases, precharging and reading, which is the same as in [6], [8]. However, for writing SRAM cells #1 and #0, the actual operations remain in the value that has been read to BL and BLB by keeping enabling sense amplifier until it has been written into SRAM cell #1 or #0. Compared with the update process of [8], the second precharging phase has been thrown away. An experiment in Section V shows that over 50% of energy is saved.

D. Pipeline Multiplier

The pipeline multiplier is illustrated in Fig. 11, which is accomplished by shift and accumulation. W[7:0] is the 8-bit weight of neural networks coming from the ping-pong SRAM block. Din is the inverting INA coming from the INA preprocessing unit. Din_d1 is the one-cycle delay version of Din. The delay component of Din is not shown in the

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:41:51 UTC from IEEE Xplore. Restrictions apply.

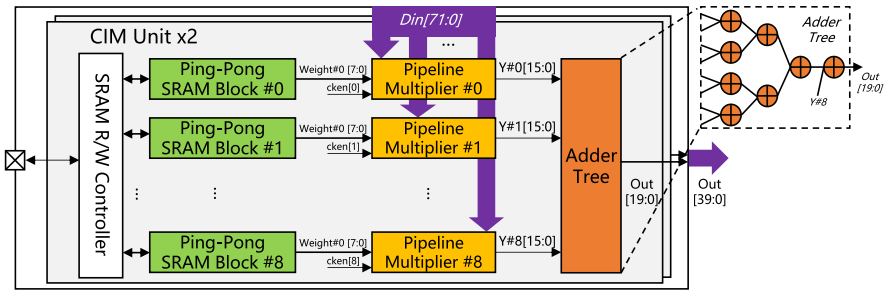


Fig. 9. Proposed CIM macro.

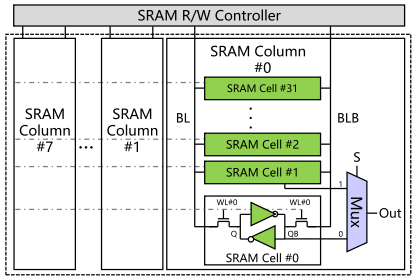


Fig. 10. Ping-pong SRAM block.

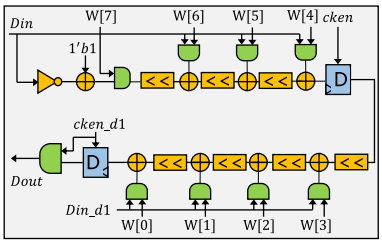


Fig. 11. Pipeline multiplier.

diagram, whose overhead can be neglected since all CIM units of the macro array share the same input. Dout is the final 16-bit multiplying result. The AND gate in the diagram is replaced by NOR gate for practical implementation, which is the reason why the QBs of SRAM cells #1 and #0 are connected to the Mux in Fig. 10. The hybrid used 14T and 28T full adder scheme of [11] is also adopted in the adders of pipeline multiplier. Only nonzero activations take part in real multiplication. When the zero index, two D flip-flops of the pipelined multiplier are clock-gated, and Dout is pulled down to zero. For example, assuming that the original INAs are $\{8'd1, 8'd2, 8'd0, 8'd3, 8'd4, 8'd0, 8'd5, 8'd0, 8'd0, 8'd6, 8'd0, 8'd7\}$, the indices are $\{12'b1101_1010_0101\}$. Din of pipeline multiplier will be $\{8'd1, 8'd2, 8'd3, 8'd4, 8'd5, 8'd6, 8'd7\}$. D flip-flops of pipeline multiplier will be clock-gated in the 3rd, 6th, 8th, 9th, and 11th cycles, and Dout will be pulled down to zero synchronously.

E. Layout Floorplan of CIM Macro

The floorplan of CIM macro is illustrated in the right of Fig. 12. The left of the diagram is the micrograph of the ping-pong SRAM block and pipeline multiplier. The layout of the pipeline multiplier is an approximate trapezoid since the bit width of the adder is growing from 8 to 16 bit. The purple rectangle presents two D flip-flops of pipeline multiplier, with

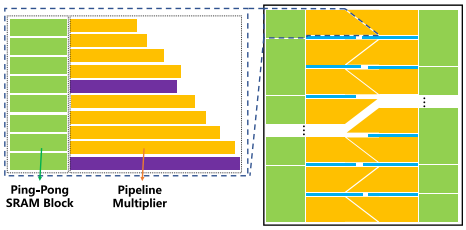


Fig. 12. Floorplan of CIM macro.

Din inputs from the short side of the trapezoid. The ping-pong SRAM block is placed at the left of the pipeline multiplier. The bit multiplication results are accumulated from top to bottom. Then, Dout is generated at the long side of the trapezoid. The different stages of the adder tree, which are denoted by blue bars in the diagram, are flattened and placed in the middle of neighboring pipeline multipliers. The convolution result is generated from the CIM unit outputs at the long side of the ninth pipeline multiplier. To optimize the area efficiency, the layout of two CIM units is placed face-to-face and inserted into each other, which is also why one CIM macro has two CIM units.

V. Evaluation and Analysis

A. Experimental Setup

The proposed CIM compute engine is validated on TSMC 22-nm ULP technology node, TT corner, 25°C. For CIM macro evaluation, the schematic and layout of CIM macro are designed manually using Cadence Virtuoso. The simulation of the CIM macro is stimulated by a digital vector file generated by an advanced programming language. The parasitic extraction and post-layout simulation are performed by StarRC and FineSim, respectively. Weights and INAs with different sparsity are tested, including completely random and 10%–90%. The whole random number is the worst case of evaluation results, which rarely occurs for practical neural networks.

For architecture evaluation, the INA preprocessing unit, pipeline accumulator, and postprocessing unit are presented by RTL code and co-simulated in VCS with a behavioral model that describes the full function of CIM macro. Synthesis and performance evaluation are finished in Design Compiler. The whole layout of PIPECIM shown in Fig. 13 was accomplished in ICC2. CIM macro is merged at this stage. The INAs and weights for the overall system and each CIM macro are

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:41:51 UTC from IEEE Xplore. Restrictions apply.

TABLE II
HARDWARE SPECIFICATION

Technology	TSMC 22nm ULP
System Area	1.44 mm^2
CIM Macro Area	$13611um^2(136.8um \times 99.5um)$
On-chip Memory	32.496 KB
INA Buffer	2.496 KB
Scratch Pad	12 KB
CIM Array SRAM	18 KB
CIM Macro SRAM	4.5 Kb
Supply Voltage	720-800 mV
Frequency	500 MHz @800mV 714 MHz @800mV (Max)
Data Precision	INT8, BF16
Power Consumption	0.22-2.6 mW (Macro)
	44.64-66.59 mW (System, INT8)
	37.41-61.53 mW (System, BF16)
System Throughput	576-1146.34 GOPS (INT8)
	144-271.26 GFPS (BF16)
Macro Area Efficiency	1.32-2.16MOPS/um^2
	5.95-9.7MOPS*Kb/um^2
Energy Efficiency	6.92-80.9 TOPS/W (Macro)
	8.69-25.68 TOPS/W (System, INT8)
	2.35-7.30 TFPS/W (System, BF16)

derived from the data of the quantized VGG-16 mentioned in Section II.

B. Hardware Specification

The hardware specification of PIPECIM is shown in Table II. The total on-chip memory is 32.49 kB, including a 2.49-kB INA buffer, 12-kB scratch pads, and an 18-kB CIM array. The SRAM size of one CIM macro is 4.5 kb. The supply voltage can be varied from 720 to 800 mV. The default working frequency is 500 MHz if not otherwise specified, which can be boosted to 714 MHz at 800 mV due to the two-stage D flip-flops inserted into CIM macro. The data precision can be selected between INT8 and BF16. A fully connected layer of neural networks can be easily implemented by dividing it into multiple Conv. 3×3 kernels. When performing compute operation at 800 mV, the power consumption of CIM macro varies from 2.6 mW at random weight and INA to 0.22 mW at 90% sparsity of weight and INA. The corresponding one is 6.92–80.9 TOPS/W. Two parameters present the area efficiency of the CIM macro: one is throughput/area and the other is (throughput $\times$ SRAM size)/area. The layout of the PIPECIM compute engine and CIM macro is shown in Fig. 13. The total area of the PIPECIM compute engine is 1.44 mm^2. Specifically, the area of each CIM macro is 13 610 μm^2 (136.8 $\times$ 99.5 μm).

C. Weight Update

The performance of the weight update scheme of the proposed ping-pong SRAM block is listed in Table III. In order to avoid the influence of design proficiency and improve the fairness of comparison, the external read SRAM step mentioned in Section IV is replaced by the read SRAM cell of CIM macro since the performance of individual SRAM buffer is optimized by the IP vendor, but our CIM macro is not. The energy-saved percentages of weight updates are more than 30% at different voltages.

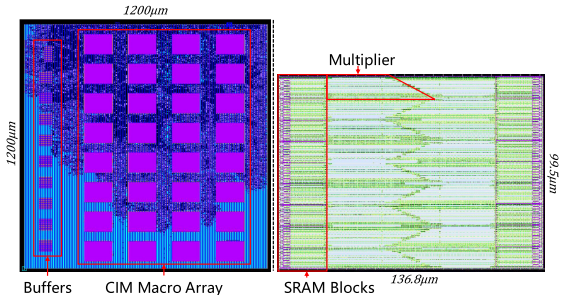
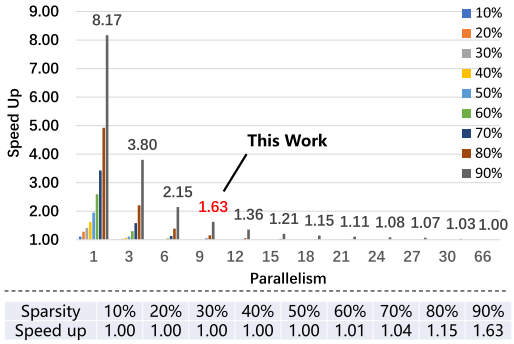


Fig. 13. Layout. Left: PIPECIM system. Right: CIM macro.

TABLE III
WEIGHT UPDATE ENERGY IMPROVEMENT OF CIM MACRO

voltage(mV)	600	650	700	750	800
from external SRAM(mA)	3.37	3.43	3.59	3.77	3.96
from SRAM cell #31∼#2(mA)	2.33	2.34	2.44	2.58	2.72
saved energy(%)	30.9	31.8	32.0	31.6	31.3



Sparsity	10%	20%	30%	40%	50%	60%	70%	80%	90%
Speed up	1.00	1.00	1.00	1.00	1.00	1.01	1.04	1.15	1.63

Fig. 14. Top: speedup performance of CIM macro at different parallelism and feature sparsity. Bottom table: detailed speedup of our proposed CIM macro at different feature sparsity.

D. Speedup and Throughput

To demonstrate the superiority of our skip strategy, we defined the total number of operations as T and the number of skipped operations as S. We define the speedup as follows:

$$\text{Speedup} = \frac{T}{T - S}. \tag{1}$$

When nine INAs of CIM macro are zero simultaneously, the corresponding computations will be skipped. The proposed CIM macro's speedup performance at different INA sparsity is shown at the bottom of Fig. 14. The 1.6$\times$ speedup has been achieved at 90% sparsity. The speedup effect fades away when the sparsity is less than 60%. In order to explore the potential speedup of each sparsity under different parallelism, that is, the size of the adder tree in the CIM macro, it has been illustrated in the histogram of Fig. 14. The speedup effect can be overlooked when parallelism is more significant than 18. Maximal speedup increases exponentially as the decrease of parallelism. When the parallelism is 1, namely, the adder tree is discarded, 8.17$\times$ speedup can be acquired at 90% INA sparsity. This phenomenon shows that the scale of adder tree must be considered seriously.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:41:51 UTC from IEEE Xplore. Restrictions apply.

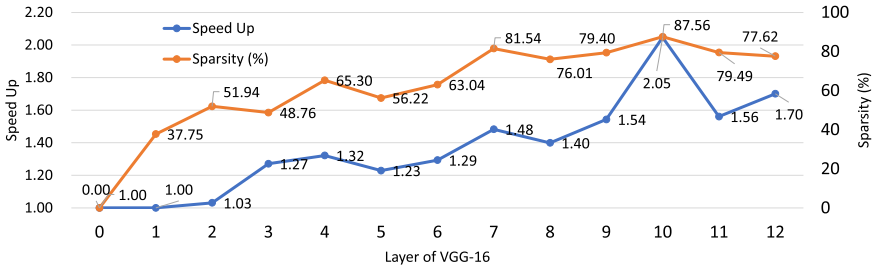


Fig. 15. Speedup performance and INA sparsity of PIPECIM compute engine on the convolution layer of VGG-16.

TABLE IV
ENERGY EFFICIENCY OF CIM MACRO

Sparsity	random	10%	20%	30%	40%	50%	60%	70%	80%	90%
Input Activation (TOPS/W)	6.92	7.14	7.76	8.33	9.62	11.3	13.4	18.6	26.2	34.8
Weight (TOPS/W)		7.19	7.98	8.75	11	13.2	17.7	21.8	26.8	37.5
Input Activation + Weight (TOPS/W)		7.33	9.04	10.8	15.3	20.3	31.7	43.9	60.1	80.9

TABLE V
COMPARISON WITH THE OTHER PRIOR WORKS

	ISSCC'21-16.4 [11]	TVLSI'21-29 [19]	VLSI'21 [20]	TVLSI'23-31 [21]	TCAS-I'22-69 [22]	TCAS-II'24-70 [23]	Our Work
Technology	22nm	TSMC 40nm	28nm	55nm	28nm	40nm	22nm
INT Precision (INA/W)	4b/4b,8b/8b	4b	N/A	4b/4b	16b	(1-16b)/(1-16b)	8b
Simultaneous Compute and Weight Update	No	No	No	No	N/A	No	Yes
Methodology	CIM Macro	CIM Macro	CIM System	CIM System	RTL System	CIM Macro	CIM System
Implementation	Silicon	Layout	Silicon	Silicon	Layout	Layout	Layout
Float Support	No	No	Yes	No	No	No	Yes
Supply Voltage(V)	0.72	0.9	0.76-1.1	0.9-1.2	0.72	0.9-1.2	0.72-0.8
Frequency(MHz)	500	100	250	65.4-187.1	700	625	500
Area(mm^2)	0.202(macro)	0.2208(macro)	5.832(system)	0.067(macro)	2.145(system)	0.41(macro)	0.0136(macro) 1.44(system)
CIM SRAM(Kb)	64	1	N/A	4	N/A	128	72
Total SRAM (KB)	N/A	N/A	396	N/A	480	N/A	53.5
Throughput (GOPS or GFLOPS)	917(INT8)	5.594(INT4)	119.4-662.0(BF16)	N/A	403(INT16)	9600(INT1) 37.5(INT16)	576-1146(INT8) 144-271.26(BF16)
CIM Macro Energy Efficiency (TOPS/W)	89(INT4)	7.66(INT4)	N/A	N/A	N/A	497(INT1) 1.94(INT16)	6.92-80.9(INT8)
System Energy Efficiency (TOPS/W or TFPS/W)	N/A	N/A	2.75(INT8)	51.4(INT4)	0.76-7.3(BF16)	2.16(INT16)	8.68-25.67(INT8) 2.35-7.30(BF16)

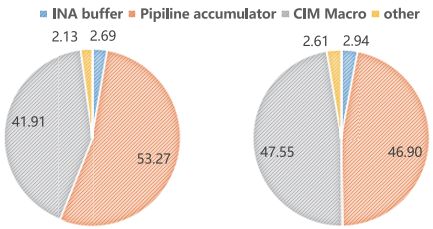


Fig. 16. Power breakdown of PIPECIM at Layer 11 of VGG-16. Left: INT8. Right: BF16.

The system speedup is shown in Fig. 15. The peak 1.94× speedup is achieved in Layer 11 with 87.56% sparsity. The speedup effect vanishes in Layers 1 and 2. Generally, the speedup is more effective at higher sparsity, but they are not a simple linear relationship. The abnormal conditions occur in Layers 4 and 13. In the meanwhile, the system speedup is better than the simulation results in Fig. 14, whose speedup is 1.63× at 90% sparsity and disappears when sparsity is less than 50%. The reason is that speedup is influenced by sparsity and the distribution of zero and non-zero values. When zero values are aggregated, skipping zero operation can be satisfied more quickly. When zero values are randomly distributed with non-zero values, skipping zero operation is hardly executed.

The INT8 throughput is 576∼1146.34 TOPS. The BF16 throughput is 144∼271.26 TFPS. The latter is approximately 1/4 of the former. This is because 2× CIM PE and time are needed for one float operation compared with INT8.

E. Energy Efficiency

The energy efficiency of CIM macro is illustrated in Table IV. The supply voltage is 800 mV. The energy efficiency is 6.92 TOPS/W when both INA and Weight are random, which can be seen as the worst working condition of CIM macro. Then, energy efficiency is improved to 34.8 or 37.5 TOPS/W when the sparsity of INA or weight increases to 90%. If both weight and INA have 90% sparsity, 80.9 TOPS/W, larger than the sum of 34.8 and 37.5, can be

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:41:51 UTC from IEEE Xplore. Restrictions apply.

achieved, indicating the significant influence of sparsity on the energy efficiency of the CIM circuit.

The energy efficiency of the overall system is measured at 720 mV, which includes the energy consumption of all components. To shorten simulation time, we only measure specific layers in which the best and worst energy efficiency may occur according to the sparsity and speedup outcome shown in Fig. 15. For INT8, the peak energy efficiency is 25.6776 TOPS/W at Layer 11 with 87.56% INA sparsity and 19.51% weight sparsity. The worst energy efficiency is 8.6887 TOPS/W at Layer 01, no sparsity is found. The power breakdown for INT8 at Layer 11 is shown in the left of Fig. 16. The CIM macro array occupies 41.91% of the total power.

At the same time, the pipeline accumulator consumes 53.27% power, higher than the former, which means that the power overhead of the accumulation step must be taken seriously for MAC operation. Read and write scratch pads are executed at this step. A larger scratch pad means higher reuse of weights stored in SRAM cell #0 or #1 in the ping-pong SRAM block, but the power overhead of accumulation increases consequently. Since all CIM macros share the same INAs, the power consumption of the INA buffer is negligible, only 2.69%. The other in Fig. 16 includes INA buffer control, postprocessing units, and so on. The difference in power breakdown at other layers is slight. The power of CIM macro array and pipeline accumulators is still dominant. For BF16, the energy efficiency is 2.35∼7.30 TFPS/W. The power breakdown of BF16 is shown in the right of Fig. 16. The share of CIM macro array and pipeline accumulators change to 47.55% and 46.9%, respectively, since only half of the pipeline accumulators is enabled.

In the comparative analysis presented in Table V, our architecture emerges distinctly by supporting floating-point operations and uniquely facilitating simultaneous computation and weight updates, a feature not available in other cited works. This capability ensures continuous processing without the delays typically associated with weight adjustments, significantly enhancing computational efficiency. Moreover, our design achieves the smallest macro area, which is a testament to the innovative engineering behind our compact layout. Notably, our use of zero-skipping techniques further boosts throughput, giving our system a competitive edge over others. These advancements collectively underscore our architecture's superior performance and efficiency, making it a groundbreaking contribution to the field of computational architecture.

VI. Conclusion

The PIPECIM solved the compute stall of conventional CIM circuits by the proposed ping-pong SRAM block and the pipeline accumulator. Besides, the proposed PIPECIM can achieve the low energy consumption of weight update and sufficient buffering ability between CIM circuit and off-chip memory. We developed a comprehensive zero-skipping and clock-gating scheme to improve the throughput and energy efficiency of the overall system according to the weight and feature sparsity of the neural network. Based on our evaluation, the energy efficiency of CIM macro is 6.92∼80.9 TOPS/W@INT8, and the system energy efficiency is 8.65∼25.67 TOPS/W@INT8 and 2.35∼7.30 TFPS/W@BF16. The system evaluation for system-on-chip (SoC)-level circuit and end-to-end implementation is our future work.

References

[1] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," 2014, *arXiv:1409.1556*.

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

[3] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 779–788.

[4] A. Vaswani et al., "Attention is all you need," in *Proc. Conf. NIPS*, Long Beach, CA, USA, Dec. 2017, pp. 6000–6010.

[5] M. Bavarian et al., "Efficient training of language models to fill in the middle," 2022, *arXiv:2207.14255*.

[6] Y.-H. Chen, T. Krishna, J. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, Jan. 2017.

[7] Z. Yuan et al., "STICKER: An energy-efficient multi-sparsity compatible accelerator for convolutional neural networks in 65-nm CMOS," *IEEE J. Solid-State Circuits*, vol. 55, no. 2, pp. 465–477, Feb. 2020.

[8] J. Yue et al., "15.2 A 2.75-to-75.9TOPS/W computing-in-memory NN processor supporting set-associate block-wise zero skipping and ping-pong CIM with simultaneous computation and weight updating," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, Feb. 2021, pp. 238–240.

[9] J.-W. Su et al., "16.3 A 28nm 384kb 6T-SRAM computation-in-memory macro with 8b precision for AI edge chips," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 250–252.

[10] L. Chang, P. Zhao, and J. Zhou, "Pipelined computing-in-memory macro with computation-memory aware technique," in *Proc. IEEE Asia–Pacific Conf. Circuits Syst. (APCCAS)*, Nov. 2022, pp. 505–509.

[11] Y.-D. Chih et al., "16.4 an 89TOPS/W and 16.3TOPS/mm^2 all-digital SRAM-based full-precision compute-in memory macro in 22nm for machine-learning edge applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 64, Feb. 2021, pp. 252–254.

[12] B. Yan et al., "A 1.041-Mb/MM 2 27.38-TOPS/W signed-INT8 dynamic-logic-based ADC-less SRAM compute-in-memory macro in 28nm with reconfigurable bitwise operation for AI and embedded applications," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Sep. 2022, pp. 188–190.

[13] H. Zhu et al., "COMB-MCM: Computing-on-memory-boundary NN processor with bipolar bitwise sparsity optimization for scalable multi-chiplet-module edge machine learning," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 1–3.

[14] H. Fujiwara et al., "A 5-nm 254-TOPS/W 221-TOPS/mm^2 fully-digital computing-in-memory macro supporting wide-range dynamic-voltage-frequency scaling and simultaneous MAC and write operations," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 1–3.

[15] P.-C. Wu et al., "A 28nm 1Mb time-domain computing-in-memory 6T-SRAM macro with a 6.6ns latency, 1241GOPS and 37.01TOPS/W for 8b-MAC operations for edge-AI devices," in *IEEE Int. Solid-State Circuits Conf. (ISSCC) Dig. Tech. Papers*, vol. 65, Feb. 2022, pp. 1–3.

[16] S. Feng et al., "MeNDA: A near-memory multi-way merge solution for sparse transposition and dataflows," in *Proc. 49th Annu. Int. Symp. Comput. Archit.*, Jun. 2022, pp. 245–258, doi: 10.1145/3470496.3527432.

[17] S. Han et al., "EIE: Efficient inference engine on compressed deep neural network," in *Proc. ACM/IEEE Int. Symp. Comput. Archit. (ISCA)*, Seoul, South Korea, Jun. 2016, pp. 243–254.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:41:51 UTC from IEEE Xplore. Restrictions apply.

[18] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 448–456.
[19] C.-C. Wang, L. K. S. Tolentino, C.-Y. Huang, and C.-H. Yeh, "A 40-nm CMOS multifunctional Computing-in-Memory (CIM) using single-ended disturb-free 7T 1-Kb SRAM," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 29, no. 12, pp. 2172–2185, Dec. 2021.
[20] J. Lee et al., "A 13.7 TFLOPS/W floating-point DNN processor using heterogeneous computing architecture with exponent-computing-in-memory," in *Proc. Symp. VLSI Circuits*, Jun. 2021, pp. 1–2.
[21] Z. Lin et al., "A fully digital SRAM-based four-layer in-memory computing unit achieving multiplication operations and results store," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 31, no. 6, pp. 776–788, Jun. 2023.
[22] Z. Shao et al., "Memory-efficient CNN accelerator based on interlayer feature map compression," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 69, no. 2, pp. 668–681, Feb. 2022.
[23] C. Zhang, M. Wang, Y. Mai, C. Tang, and Z. Yu, "A high-density and reconfigurable SRAM-based digital compute-in-memory macro for low-power AI chips," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 70, no. 9, pp. 3589–3593, Sep. 2023.



Yuanbo Wang received the B.E. degree from the University of Electronic Science and Technology of China (UESTC), Chengdu, China, in 2023, where he is currently working toward the M.S. degree at the Department of Internet of Things Engineering, under the guidance of Prof. Liang Chang.

His current research interests include artificial intelligence (AI) accelerators and computing-in-memory circuits.



Liang Chang (Member, IEEE) received the double B.S. degree from Chengdu University of Information and Technology, Chengdu, China, and the University of Electronic Science and Technology of China, Chengdu, in 2011, and the M.S. and Ph.D. degrees from Beihang University, Beijing, China, in 2014 and 2019, respectively.

He was an Engineer with China Glorun Technology and AMD China, Beijing, from 2012 to 2015. Since 2020, he has been an Associate Professor at the School of Information and Communication Engineering, University of Electronic Science and Technology of China. He was a Visiting Scholar at the University of California at Santa Barbara (UCSB), Santa Barbara, CA, USA, and The Hong Kong University of Science and Technology (HKUST), Hong Kong, in 2017 and 2022, respectively. He has published more than 60 scientific papers in prestigious conferences and journals, including IEEE International Solid-State Circuits Conference (ISSCC), MICRO, DAC, IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—I: REGULAR PAPERS (TCAS-I), IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—II: EXPRESS BRIEFS (TCAS-II), IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION (VLSI) SYSTEMS, and IEEE TRANSACTIONS ON COMPUTER-AIDED DESIGN OF INTEGRATED CIRCUITS AND SYSTEMS (TCAD). His research interests include computing in emerging nonvolatile memory, advanced memory-centric computer architecture, and artificial intelligence (AI) processors for intelligent detection.

Dr. Chang is a regular reviewer of IEEE JOURNAL OF SOLID-STATE CIRCUITS, IEEE TCAS-I, IEEE TCAS-II, IEEE TCAD, IEEE TRANSACTIONS ON COMPUTERS, IEEE TRANSACTIONS ON BIOMEDICAL CIRCUITS AND SYSTEMS, and IEEE TVLSI, and a Guest Editor of *Electronics*.



Jingke Wang is currently working toward the B.E. degree at the Department of Internet of Things Engineering, School of Information and Communication Engineering, University of Electronic Science and Technology of China (UESTC), Chengdu, China.

Her current research interests include computing-in-memory circuits.



Pan Zhao (Student Member, IEEE) received the B.E. degree from Sichuan University, Chengdu, China, in 2022. He is currently working toward the M.S. degree at the Department of Internet of Things Engineering, School of Information and Communication Engineering, University of Electronic Science and Technology of China, Chengdu, under the guidance of Prof. Liang Chang.

He won the National Undergraduate Scholarship in 2021. His research interests include computing-in-memory circuits.



Jiahao Zeng received the B.E. degree from the University of Electronic Science and Technology of China (UESTC), Chengdu, China, in 2023, where he is currently working toward the M.S. degree at the Department of Internet of Things, under the guidance of Prof. Liang Chang.

His current research interests include computing-in-memory (CIM) and super-resolution.



Xin Zhao (Student Member, IEEE) received the B.E. degree in electronic engineering from China University of Mining and Technology, Xuzhou, China, in 2021. He is currently working toward the M.S. degree at the School of Information and Communication Engineering, University of Electronic Science and Technology of China, Chengdu, China, under the supervision of Prof. Liang Chang.

He was a Visiting Student at The Hong Kong University of Science and Technology, Hong Kong, in 2023. His research interests include hardware-algorithm co-design for artificial intelligence (AI) accelerators, super-resolution, and computing-in-memory (CIM) architecture.



Wuyang Hao received the master's degree from Tsinghua University, Beijing, China, in 1994, and the University of Michigan, Ann Arbor, MI, USA, in 1997.

He was a Lead Engineer and the Vice President of various companies, including AMD, Beijing/SPANSION, Sunnyvale, CA, USA, MICRON, and QUALCOMM, San Jose, CA, USA. He is an Engineer at the University of Electronic Science and Technology of China (UESTC), Chengdu, China. He holds more than 20 patents on magnetic random access memory. His research interests are nonvolatile memory and computing-in memory.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:41:51 UTC from IEEE Xplore. Restrictions apply.



Liang Zhou received the Ph.D. degree from the University of Electronic Science and Technology of China (UESTC), Chengdu, China, in 2014.

He is an Associate Professor at UESTC. His research interests include neural network accelerators, wireless sensor networks, AIoT, and signal processing in communication systems.



Haining Tan received the Ph.D. degree in computer science and technology from the Institute of Computing Technology (ICT), Chinese Academy of Sciences (CAS), Beijing, China, in 2022.

He is an Assistant Professor at ICT, CAS. His current research interests include computer architecture and computational intelligence.



Yinhe Han (Senior Member, IEEE) received the M.S. and Ph.D. degrees in computer science from the Institute of Computing Technology (ICT), Chinese Academy of Sciences (CAS), Beijing, China, in 2003 and 2006, respectively.

He is currently a Professor with ICT, CAS. His main research interests are microprocessor design, integrated circuit design, and computer architecture.



Jun Zhou (Senior Member, IEEE) received the dual B.S. degree in communication engineering and microelectronics from the University of Electronic Science and Technology of China (UESTC), Chengdu, China, in 2004, and the Ph.D. degree in microelectronics system design from Newcastle University, Newcastle upon Tyne, U.K., in 2008.

He is currently a Professor at UESTC. Before joining UESTC, he was a Research Scientist and a Principal Investigator at IMEC Netherlands, Eindhoven, The Netherlands; and the Institute of Microelectronics, A*STAR, Singapore, from 2008 to 2017. He has published more than 100 papers in prestigious conferences and journals, including ISSCC, IEEE JOURNAL OF SOLID-STATE CIRCUITS, DAC, CICC, IEEE TRANSACTIONS ON BIOMEDICAL CIRCUITS AND SYSTEMS, and IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS—I: REGULAR PAPERS. His major research interests include algorithm and processor co-design for low-power intelligent sensing.

Dr. Zhou served as a TPC/OC Member for a number of IEEE conferences, including DAC, SOCC, ICCD, and ISCAS. He is serving as the A-SSCC TPC Sub-Committee Chair for Digital Circuits and Systems and an Associate Editor for IEEE TRANSACTIONS ON BIOMEDICAL CIRCUITS AND SYSTEMS and IEEE TRANSACTIONS ON VERY LARGE SCALE INTEGRATION (VLSI) SYSTEMS.

Authorized licensed use limited to: University of Shanghai For Science and Technology. Downloaded on September 29,2026 at 13:41:51 UTC from IEEE Xplore. Restrictions apply.